🚀 FAZ 6.5 — Keyword Mapping Doldurma (Başlıyoruz)

Aşağıda **tam ve çalışır** bir script veriyorum. Bu script, 42 metriğin keyword eşleşmelerini TMDB API'den otomatik çekecek ve `data/taste/keyword-mappings.json` dosyasına yazacak.

📁 ADIM 1: `scripts/fetch-keyword-ids.mjs` Oluştur

```javascript
#!/usr/bin/env node
//
// ═══════════════════════════════════════════════════════════════════
// CineTrack Taste Engine — Keyword ID Fetcher (Faz 6.5)
//
// 42 metriğin keyword eşleşmelerini TMDB API'den otomatik çeker.
// Sonuçları:
//   1. data/taste/keyword-mappings.json  (validate-real-data.mjs'in kullanacağı dosya)
//   2. scripts/keyword-verification.csv  (manuel doğrulama için)
//
// MUTLAK KURALLAR:
//   1. data/cinetrack-data.json ASLA yazılmaz.
//   2. TMDB API Key hardcode edilmez; process.env.TMDB_API_KEY veya .env'den okunur.
//   3. Rate-limit koruması: 250ms delay + 429 handling.
//   4. Çakışan keyword'ler raporlanır, sessizce atlanmaz.
//
// ═══════════════════════════════════════════════════════════════════
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const PROJECT_ROOT = path.resolve(__dirname, "..");

const TMDB_BASE = "https://api.themoviedb.org/3";
const DELAY_MS = 300; // Rate-limit koruması (TMDB 40 req/10s)

//
// ═══════════════════════════════════════════════════════════════════
// METRİK → KEYWORD ADAYLARI EŞLEMESİ
// Bu liste, taksonomideki "evidenceRequirement" alanından türetilmiştir.
// Her metrik için birden fazla keyword adayı olabilir.
```

```
//
================================================================================
=====
const METRIC_KEYWORD_CANDIDATES = {
  // === MOOD (6) ===
  "mood.melancholic": ["melancholy", "grief", "sorrow", "sadness", "melancholia", "mourning"],
  "mood.dark": ["dark", "bleak", "grim", "dystopian", "darkness", "gloomy"],
  "mood.hopeful": ["hope", "redemption", "uplifting", "feel-good", "hopeful", "inspiring"],
  "mood.tense": ["tension", "suspense", "anxiety", "edge of seat", "nerve-racking"],
  "mood.cozy": ["heartwarming", "slice of life", "comforting", "cozy", "warm"],
  "mood.unsettling": ["disturbing", "creepy", "psychological", "eerie", "uneasy",
"uncomfortable"],

  // === NARRATIVE (3 aktif) ===
  "narrative.non_linear": ["nonlinear", "time travel", "flashback", "multiple timelines",
"non-linear"],
  "narrative.open_ending": ["ambiguous ending", "open ending", "ambiguous", "open-ended"],
  "narrative.twist": ["twist ending", "plot twist", "surprise ending", "twist", "unexpected"],

  // === CONCEPT (6) ===
  "concept.identity": ["identity", "clone", "doppelganger", "memory loss", "amnesia", "identity
crisis"],
  "concept.mortality": ["death", "aging", "terminal illness", "afterlife", "mortality", "dying"],
  "concept.technology": ["artificial intelligence", "robot", "cyber", "hacking", "ai", "technology"],
  "concept.freedom": ["oppression", "rebellion", "revolution", "escape", "freedom",
"liberation"],
  "concept.love": ["love", "romance", "relationship", "family bonds", "love story"],
  "concept.power": ["politics", "corruption", "empire", "dictatorship", "power", "political"],

  // === CHARACTER (2 aktif) ===
  "character.antihero": ["antihero", "villain protagonist", "morally grey", "anti-hero", "dark
hero"],
  "character.solitary": ["isolation", "loneliness", "lone wolf", "survival", "solitary", "alone"],

  // === STYLE (2 aktif) ===
  "style.surreal": ["surreal", "dreamlike", "hallucination", "abstract", "surrealism"],
  "style.stylized": ["neon", "stylized", "visually striking", "cyberpunk", "stylized"],

  // === SETTING (3 aktif) ===
  "setting.dystopia": ["dystopia", "post-apocalyptic", "wasteland", "totalitarian", "dystopian"],
  "setting.space": ["space", "spaceship", "astronaut", "alien planet", "outer space"],
  "setting.historical": ["historical", "period", "19th century", "medieval", "war", "period drama"],

  // === TV'de keyword'den türetilen genre'lar (3) ===
  "genre.thriller": ["thriller", "suspense", "psychological thriller"],
  "genre.horror": ["horror", "scary", "terror", "frightening"],
  "genre.romance": ["romance", "romantic", "love story"],
};
```

```
//
// ORTAM / API KEY
//
function loadEnvFile(envPath) {
  if (!fs.existsSync(envPath)) return {};
  const out = {};
  for (const line of fs.readFileSync(envPath, "utf8").split(/\r?\n/)) {
    const trimmed = line.trim();
    if (!trimmed || trimmed.startsWith("#")) continue;
    const eq = trimmed.indexOf("=");
    if (eq === -1) continue;
    const key = trimmed.slice(0, eq).trim();
    let value = trimmed.slice(eq + 1).trim();
    if ((value.startsWith("") && value.endsWith("")) ||
        (value.startsWith(""") && value.endsWith("""))) {
      value = value.slice(1, -1);
    }
    out[key] = value;
  }
  return out;
}

function resolveApiKey() {
  if (process.env.TMDB_API_KEY) return process.env.TMDB_API_KEY;
  const envPath = path.join(PROJECT_ROOT, ".env");
  const fromFile = loadEnvFile(envPath).TMDB_API_KEY;
  if (fromFile) return fromFile;
  console.error(
    "HATA: TMDB_API_KEY bulunamadı.\n" +
    "  a) node --env-file=.env scripts/fetch-keyword-ids.mjs ile çalıştırın, veya\n" +
    "  b) .env dosyasında TMDB_API_KEY satırı olduğundan emin olun."
  );
  process.exit(1);
}

//
// TMDB İSTEMCİSİ
//
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
```

```
async function fetchTmdb(endpoint, apiKey) {
  const url = new URL(`TMDB_BASE{endpoint}`);
  url.searchParams.set("api_key", apiKey);

  for (let attempt = 1; attempt <= 4; attempt++) {
    try {
      const res = await fetch(url, {
        headers: { Accept: "application/json" },
        signal: AbortSignal.timeout(15000),
      });

      if (res.status === 429) {
        const retryAfter = Number(res.headers.get("retry-after")) || 2;
        console.log(`  ⏳ Rate limit, ${retryAfter}s bekleniyor...`);
        await sleep(retryAfter * 1000);
        continue;
      }

      if (res.status === 401 || res.status === 403) {
        console.error(`HATA: TMDB yetkilendirme hatası (HTTP ${res.status}).`);
        process.exit(1);
      }

      if (res.status === 404) return { status: 404, data: null };
      if (!res.ok) {
        await sleep(attempt * 500);
        continue;
      }

      const data = await res.json();
      return { status: 200, data };
    } catch (err) {
      await sleep(attempt * 500);
    }
  }
  return { status: 0, data: null };
}

async function searchKeyword(query, apiKey) {
  const res = await
fetchTmdb(`/search/keyword?query=${encodeURIComponent(query)}&page=1`, apiKey);
  await sleep(DELAY_MS);

  if (res.status !== 200 || !res.data?.results?.length) {
    return null;
  }
```

```
  // En yakın eşleşmeyi bul (tam eşleşme öncelikli)
  const results = res.data.results;
  const exact = results.find((r) => r.name.toLowerCase() === query.toLowerCase());
  if (exact) return exact;

  // Yoksa ilk sonucu döndür
  return results[0];
}

//
================================================================================
=====
// ANA AKIŞ
//
================================================================================
=====
async function main() {
  const apiKey = resolveApiKey();
  console.log("🔑 TMDB API Key yüklendi.");

  // 1) Taksonomiyi oku
  const taxonomyPath = path.join(PROJECT_ROOT, "data", "taste",
"cinetrack-taste-taxonomy.json");
  if (!fs.existsSync(taxonomyPath)) {
    console.error(`HATA: Taksonomi dosyası bulunamadı: ${taxonomyPath}`);
    process.exit(1);
  }
  const taxonomyDoc = JSON.parse(fs.readFileSync(taxonomyPath, "utf8"));
  const metrics = taxonomyDoc.metrics || [];
  console.log(`📚 Taksonomi yüklendi: ${metrics.length} metrik`);

  // 2) Her metrik için keyword ara
  const keywordToMetric = {}; // keywordId -> metricId
  const metricToKeywords = {}; // metricId -> [{keywordId, keywordName, query}]
  const conflicts = []; // Aynı keyword birden fazla metriğe eşleşirse
  const notFound = []; // Keyword bulunamayan metrikler
  const csvRows = [["metricId", "keywordId", "keywordName", "query", "status", "notes"]];

  let totalCalls = 0;
  let totalFound = 0;

  console.log(`\n🔍 ${Object.keys(METRIC_KEYWORD_CANDIDATES).length} metrik için
keyword aranıyor...\n`);

  for (const [metricId, candidates] of Object.entries(METRIC_KEYWORD_CANDIDATES)) {
    console.log(`[${metricId}] ${candidates.length} aday aranıyor...`);
```

```
if (!metricToKeywords[metricId]) metricToKeywords[metricId] = [];

let foundForMetric = 0;

for (const query of candidates) {
  totalCalls++;
  const result = await searchKeyword(query, apiKey);

  if (!result) {
    csvRows.push([metricId, "", "", query, "NOT_FOUND", ""]);
    continue;
  }

  const keywordId = result.id;
  const keywordName = result.name;

  // Çakışma kontrolü
  if (keywordToMetric[keywordId]) {
    const existingMetric = keywordToMetric[keywordId];
    if (existingMetric !== metricId) {
      conflicts.push({
        keywordId,
        keywordName,
        metrics: [existingMetric, metricId],
      });
      csvRows.push([metricId, keywordId, keywordName, query, "CONFLICT",
        `Aynı keyword ${existingMetric} ile de eşleşiyor`]);
      continue;
    }
  }

  // Kaydet
  keywordToMetric[keywordId] = metricId;
  metricToKeywords[metricId].push({
    keywordId,
    keywordName,
    query,
  });
  foundForMetric++;
  totalFound++;

  csvRows.push([metricId, keywordId, keywordName, query, "OK", ""]);
  console.log(`  ✓ "${query}" → ID keywordId({keywordName})`);
}

if (foundForMetric === 0) {
  notFound.push(metricId);
  console.log(`  ✗ Hiçbir aday bulunamadı`);
```

```
    } else {
      console.log(`  → foundForMetric/{candidates.length} bulundu\n`);
    }
  }

  // 3) JSON çıktısı üret (validate-real-data.mjs'in kullanacağı format)
  const outputPath = path.join(PROJECT_ROOT, "data", "taste", "keyword-mappings.json");
  const outputDoc = {
    schemaVersion: "1.0.0",
    generatedAt: new Date().toISOString(),
    tmdbMappingVersion: "tmdb-2026-11-v2",
    taxonomyVersion: taxonomyDoc.taxonomyVersion,
    totalMetrics: Object.keys(METRIC_KEYWORD_CANDIDATES).length,
    totalKeywordsFound: totalFound,
    totalCalls,
    conflicts: conflicts.length,
    notFoundMetrics: notFound,
    KEYWORD_TO_METRIC: keywordToMetric, // { "1234": "mood.melancholic" }
    METRIC_TO_KEYWORDS: metricToKeywords, // { "mood.melancholic": [{keywordId,
keywordName, query}] }
  };

  fs.writeFileSync(outputPath, JSON.stringify(outputDoc, null, 2));
  console.log(`\n✅ JSON kaydedildi: ${outputPath}`);

  // 4) CSV raporu üret (manuel doğrulama için)
  const csvPath = path.join(PROJECT_ROOT, "scripts", "keyword-verification.csv");
  const csvContent = csvRows.map((row) => row.map((c) => `"${String(c).replace(/"/g,
'""')}"`).join(",")).join("\n");
  fs.writeFileSync(csvPath, csvContent);
  console.log(`✅ CSV raporu kaydedildi: ${csvPath}`);

  // 5) Özet rapor
  console.log(`\n${"=".repeat(70)}`);
  console.log("ÖZET RAPOR");
  console.log("=".repeat(70));
  console.log(`  Toplam TMDB çağrısı: ${totalCalls}`);
  console.log(`  Bulunan keyword: ${totalFound}`);
  console.log(`  Çakışma: ${conflicts.length}`);
  console.log(`  Hiç bulunamayan metrik: ${notFound.length}`);

  if (conflicts.length > 0) {
    console.log(`\n⚠️  ÇAKIŞMALAR (manuel inceleme gerekli):`);
    for (const c of conflicts) {
      console.log(`  • Keyword "${c.keywordName}" (ID ${c.keywordId}): ${c.metrics.join(" ↔
")}`);
    }
  }
```

```
  if (notFound.length > 0) {
    console.log(`\n⚠️  BULUNAMAYAN METRİKLER (keyword adayları TMDB'de yok):`);
    for (const m of notFound) {
      console.log(`  • ${m}`);
    }
  }

  console.log(`\n${"=".repeat(70)}`);
  console.log("SONRAKİ ADIMLAR:");
  console.log("=".repeat(70));
  console.log("1. scripts/keyword-verification.csv dosyasını aç ve CONFLICT/NOT_FOUND satırlarını incele.");
  console.log("2. Gerekirse METRIC_KEYWORD_CANDIDATES listesini düzenle ve scripti tekrar çalıştır.");
  console.log("3. Her şey doğruysa, validate-real-data.mjs'i güncelle:");
  console.log(`    const KEYWORD_TO_METRIC = JSON.parse(fs.readFileSync("data/taste/keyword-mappings.json")).KEYWORD_TO_METRIC;`);
  console.log("4. validate-real-data.mjs'i tekrar çalıştır:");
  console.log(`    node --env-file=.env scripts/validate-real-data.mjs --sample-size 50 --max-calls 50`);
  console.log("=".repeat(70));
}

main().catch((err) => {
  console.error("BEKLENMEYEN HATA:", err);
  process.exit(1);
});
```

📋 ADIM 2: Çalıştırma Talimatı

Terminalde şu komutu çalıştır:

```bash
node --env-file=.env scripts/fetch-keyword-ids.mjs
```

Beklenen çıktı:
- `data/taste/keyword-mappings.json` — Keyword ID eşlemeleri
- `scripts/keyword-verification.csv` — Manuel doğrulama için CSV
- Terminalde özet rapor (çakışmalar, bulunamayan metrikler)

Süre: ~42 metrik × ~4 aday = ~168 TMDB çağrısı × 300ms = ~50 saniye

---

📋 ADIM 3: Manuel Doğrulama

Script bittikten sonra `scripts/keyword-verification.csv` dosyasını aç:

1. **CONFLICT satırları:** Aynı keyword birden fazla metriğe eşleşmiş. Örneğin "artificial intelligence" hem `concept.technology` hem `concept.consciousness` için bulunabilir. Bu durumda:
   - Ya iki metriğe de eşle (çoktan çoğa eşleme)
   - Ya da daha spesifik bir keyword seç (örn: "artificial intelligence" → technology, "consciousness" → consciousness)

2. **NOT_FOUND satırları:** Keyword TMDB'de yok. Aday listesini değiştir (örn: "feel-good" yerine "uplifting").

3. **OK satırları:** Doğru görünüyor, dokunma.

📋 ADIM 4: `validate-real-data.mjs`'i Güncelle

`validate-real-data.mjs` dosyasında şu satırı bul:

```javascript
const KEYWORD_TO_METRIC = {};
```

Şununla değiştir:

```javascript
// Keyword mapping'i JSON'dan yükle (Faz 6.5 çıktısı)
const keywordMappingsPath = path.join(PROJECT_ROOT, "data", "taste",
"keyword-mappings.json");
const KEYWORD_TO_METRIC = fs.existsSync(keywordMappingsPath)
  ? JSON.parse(fs.readFileSync(keywordMappingsPath, "utf8")).KEYWORD_TO_METRIC
  : {};
```

📋 ADIM 5: Tekrar Çalıştır

```bash
node --env-file=.env scripts/validate-real-data.mjs --sample-size 50 --max-calls 50
```

Beklenen iyileşme:

- Completeness: %25-35 → **%60-70**
- Aktif metrik sayısı: ~15-20 → **~35-37**
- Mood/concept/narrative/character/style/setting metrikleri artık kanıt üretmeli

---